



Escuela de Ingeniería en Computadores

CE3201 - Taller de Diseño Digital

Laboratorio #3 - Documento de Diseño

Estudiantes:

Mario Gudiño Rovira
Michael Valverde Navarro

Profesor:

Luis Alonso Barboza Artavia

Semestre:

II Semestre 2025

Índice

1. Repositorio en Github	2
2. Propuestas de Diseño	2
2.1. Propuesta 1	2
2.2. Propuesta 2: FSM tipo Moore monolítica (única FSM grande)	5
2.3. Propuesta 3: Diseño jerárquico con FSM principal y sub-FSMs (control distribuido)	6
2.4. Diseño elegido	7

1. Repositorio en Github

Enlace al repositorio de Github:

https://github.com/hazielgr/m_gudino_mvalverde_digital_design_lab_2025

2. Propuestas de Diseño

Para el laboratorio #3 se implementará una Máquina de Estados Finitos para controlar una partida de juego, el juego a controlar corresponde a Memoria, donde se tienen #16 cartas en un tablero desplegadas en un monitor, juegan dos personas por turnos, el jugador #1 escoge dos cartas y si estas son iguales se retiran del tablero y se suma el score de parejas encontradas en un display de 7 segmentos para el jugador uno, si el jugador uno encuentra la pareja continua jugando hasta fallar, si se falla al encontrar una pareja, se procede con el turno del jugador #2, para ambos jugadores se tienen #15 segundos paraa realizar una movida, si el tiempo se cumple la Máquina de Estados Finitos se encargará de hacer la movida por dicho jugador. El juego termina cuando todas las cartas han sido retiradas o encontradas del tablero y se muestra en pantalla cuál jugador ha ganado la partida. Existe también la posibilidad de tener un mecanismo que funcione como reset y permita iniciar desde cero el juego al accionarse.

Tomando en cuenta el funcionamiento del juego y sus reglas se presentan dos propuestas de diseño para lograr el objetivo de desarrollo.

2.1. Propuesta 1

Se presenta una FSM sencilla con los siguientes estados definidos:

- **S_INIT** El sistema se inicializa, se limpian todas las variables de control, no hay cartas emparejadas aún, no hay cartas visibles aún y los contadores de puntaje de ambos jugadores se ponen en cero. Se define que el primer turno lo inicia el Jugador 1. También se cawrga el temporizador de 15 segundos, aunque aún no corre hasta que arranca el turno. Al salir de este estado, se pasa a S_TURN_START
- **S_TURN_START** Inicio de un nuevo turno (ya sea del Jugador 1 o del Jugador 2). Se limpian los registros de las cartas seleccionadas, y se habilita el temporizador de 15 segundos. La FSM queda lista para que el jugador escoja su primera carta y por eso transiciona a S_NAV1
- **S_NAV1** El jugador mueve un cursor sobre las cartas del tablero usando botones de dirección (push buttons en la FPGA). se guarda el índice de esa carta y se pasa a S_REV1 para revelarla. Si el temporizador llega a cero antes de que el jugador escoja, la FSM pasa a S_AUTO1, el sistema elige automáticamente la primera carta.
- **S_REV1** En este estado se destapa la primera carta seleccionada. Esto se logra activando una variable o señal, que actualiza la máscara de cartas reveladas. Luego de mostrar la carta, el flujo continúa hacia S_NAV2, donde se espera la segunda selección.

- **S_NAV2** El jugador sigue navegando con el cursor, pero ahora debe elegir una segunda carta que sea distinta a la primera. Si presiona sobre una carta válida (no emparejada y diferente de la primera), se pasa a S_REV2 para revelarla. Si el tiempo de 15 segundos se agota antes de la elección, la FSM va a S_AUTO2 y el sistema selecciona automáticamente la segunda carta
- **S_REV2** Se destapa la segunda carta elegida por el jugador o seleccionada automáticamente. Al igual que en la primera, se activa el pulso que actualiza la máscara de cartas visibles. Na vez ambas cartas están destapadas, se pasa a S_EVAL para analizarlas.
- **S_EVAL** En este estado la FSM compara las dos cartas recién reveladas. Si son iguales se activa un pulso que marca esas cartas como emparejadas de manera permanente y se incrementa el contador del jugador actual. Si todas las cartas ya están emparejadas, se pasa directamente a S_GAMEOVER. Si todavía quedan cartas, el jugador mantiene el turno, se recarga el temporizador a 15 segundos, y se regresa a S_TURN_START. Si las dos cartas son distintas, se pasa al estado S_SHOWMISS para mostrar un momento el error antes de esconderlas.
- **S_SHOWMISS** En este estado se mantiene visibles las dos cartas que no coincidieron durante un pequeño intervalo (0.5–1 s), usando un mini-temporizador interno. Esto sirve para que el jugador vea cuál fue su error.
- **S_AUTO1** Si el jugador no elige la primera carta dentro del límite de tiempo, el sistema utiliza el generador pseudoaleatorio para seleccionar automáticamente una carta válida. Esa carta se revela como si el jugador la hubiese escogido.
- **S_AUTO2** Si el tiempo se agota mientras el jugador debía escoger la segunda carta, el sistema elige automáticamente una carta válida que no sea la primera.
- **S_GAMEOVER** Este es el estado final del juego. Se revisa qué jugador tiene mayor puntaje y se define el ganador, o se declara empate si los puntajes son iguales. La pantalla muestra el resultado final. La FSM se queda aquí esperando que el botón o switch RESET sea presionado.
- **Reset global** En cualquier estado si se activa la señal de RESET, la FSM vuelve inmediatamente a S_INIT, limpiando puntajes, máscaras de cartas y el temporizador. Esto garantiza que el juego pueda reiniciarse desde cero en cualquier momento

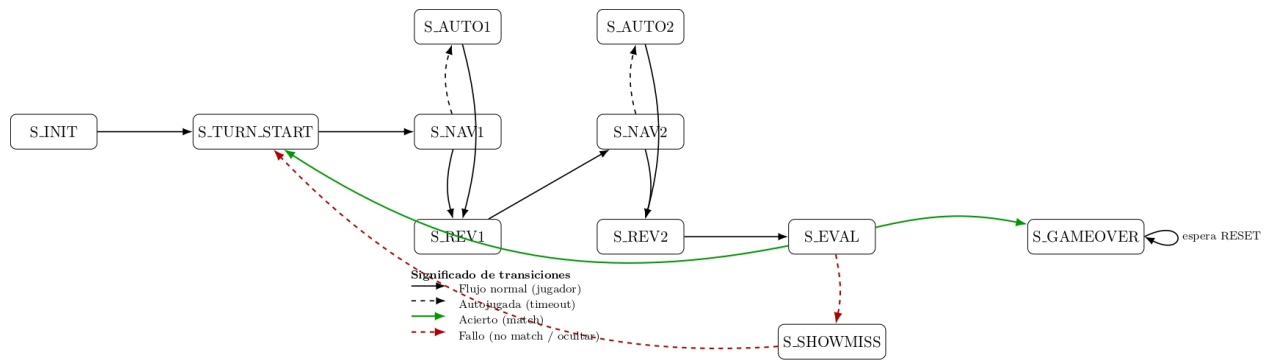


Figura 1: Propuesta 1 de FSM

En cuanto al desarrollo del controlador VGA se esperaría desplegar una interfaz gráfica en donde se tienen 16 cartas, estas cartas estarían ordenadas en forma de una matriz 4x4, de manera que los botones de la FPGA puedan servir como controles para elegir una carta, es decir, un botón se movería en la dirección de filas de la matriz, y el otro botón se movería en la dirección de columnas de la matriz, por lo que usando este mecanismo se garantiza que ambos jugadores puedan recorrer el tablero de cartas de manera libre, y se haría la elección de una carta subiendo algún switch en la FPGA.

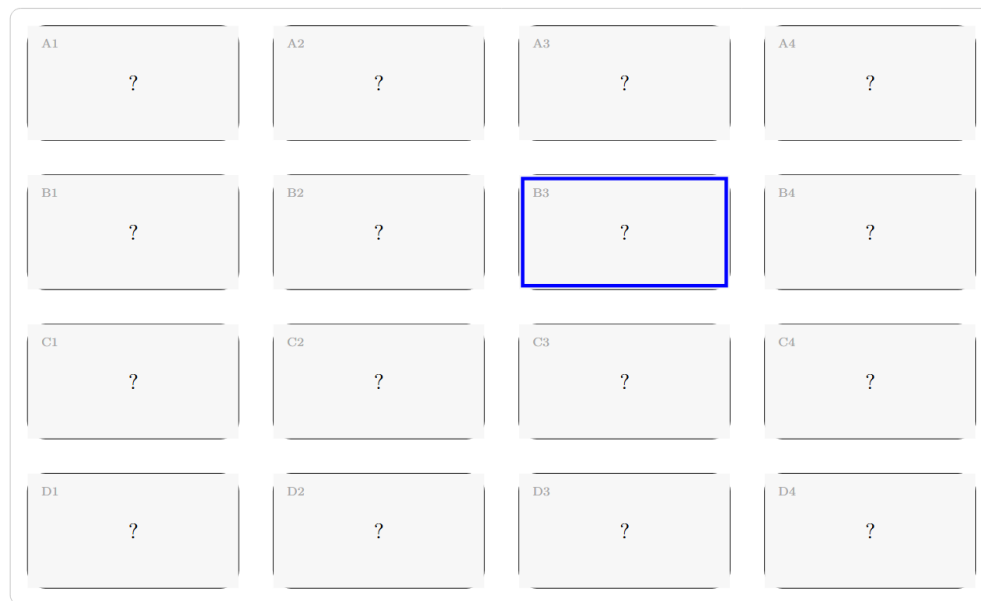


Figura 2: Diagrama de como se espera ver las cartas en pantalla desplegadas por el controlador VGA

Ventajas:

- Simple: se da un flujo lineal y fácil de seguir
- Determinismo de turnos: sigue reglas claras sin máquinas auxiliares

- Predecible para testbench: como se tiene una FSM simple, el Testbench se puede realizar de forma fácil y que sea verificable de manera rápida.

Desventajas:

- Mantenimiento de índices: se requiere registros auxiliares para llevar la cuenta de los índices de cartas y máscaras coherentes, si estos se manejan mal la UI dada por VGA quedará inconsistente.
- Más transiciones explícitas: AUTO1/AUTO2 añaden estados extra, lo cual puede ser contraproducente al desarrollar la FSM y complicar el proceso.

2.2. Propuesta 2: FSM tipo Moore monolítica (única FSM grande)

En esta propuesta se plantea un único bloque de control principal, una Máquina de Estados Finitos (FSM) que gobierna todas las funciones del juego. La FSM se encarga de coordinar los turnos de los jugadores, la selección de cartas, el conteo de tiempo, la comparación de las cartas, el cálculo de los puntajes y la indicación de ganador. Los demás módulos del sistema (controlador VGA, memoria de las cartas, temporizador de 15 segundos, generador de números aleatorios, displays de 7 segmentos) actúan únicamente como periféricos controlados, recibiendo señales de control desde la FSM y devolviendo banderas o datos simples (por ejemplo, “carta seleccionada”, “tiempo agotado”).

El flujo del juego se modela completamente dentro de esta única máquina, que recorrería estados como:

- **RESET** — inicializa, baraja cartas, limpia puntajes.
- **IDLE** — muestra tablero, espera inicio (o espera player input).
- **WAIT_DEBOUNCE** — tras pulsación, esperar señal limpia.
- **REVEAL1** — revelar primera carta (salidas: señal a VGA para mostrar).
- **WAIT_SELECT2** — esperar segunda selección (start 15s timer si necesario).
- **REVEAL2** — revelar segunda carta.
- **COMPARE** — señal de comparación; si igual → MATCH; si no → MISMATCH.
- **MATCH** — incrementar puntaje, check game end, si gana → SHOW_WINNER, sino quedarse en mismo jugador → volver a IDLE.
- **MISMATCH** — mostrar por breve periodo (p. ej. 1 s usando contador interno), luego ocultar, cambiar turno → IDLE.
- **TIMEOUT_ASSIGN** — si 15s expiran durante selección, generar índice aleatorio, simular selección.
- **SHOW_WINNER** — mostrar ganador en VGA, bloquear entradas.

En este diseño, todas las decisiones y todas las salidas críticas provienen de los estados de la FSM, lo que garantiza que la lógica de control esté concentrada en un solo punto. Esto simplifica la validación del requisito de laboratorio (“toda la lógica de control se maneja con una FSM”), ya que no hay sub-máquinas que tomen decisiones independientes.

En la figura 3 se muestra un diagrama de bloques del diseño propuesto.

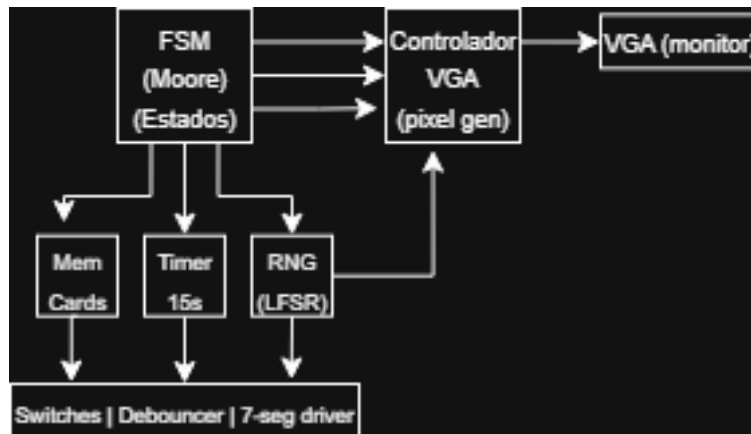


Figura 3: Diagrama de bloques FSM Moore

Ventajas:

- Menos complejidad en responsabilidades ya que un solo módulo decide todo.
- Fácil de testear en conjunto, porque se valida una sola secuencia de estados.
- Menor riesgo de inconsistencias entre módulos.

Desventajas:

- La FSM puede volverse grande y difícil de mantener, porque debe contemplar todos los casos del juego.
- Cualquier modificación (por ejemplo, cambiar el tiempo por jugada) obliga a revisar la misma máquina completa.

2.3. Propuesta 3: Diseño jerárquico con FSM principal y sub-FSMs (control distribuido)

Esta propuesta también se basa en una FSM principal, pero reparte la lógica en varios módulos especializados, cada uno con una pequeña máquina de estados propia. La FSM principal sigue siendo la encargada de las decisiones de alto nivel: cuándo empieza un turno, cuándo comparar cartas, cuándo cambiar de jugador o finalizar la partida. Sin embargo, tareas específicas se delegan a sub-FSMs que trabajan de manera autónoma una vez habilitadas por la principal.

FSM principal reduce su responsabilidad (alto nivel), delegando tareas puntuales a sub-FSMs: SelectionFSM, RevealFSM, TimeoutFSM, ScoreFSM. Uso Mealy para las sub-FSMs

donde las salidas pueden depender de entradas y estado (más reactivo). La FSM principal coordina y genera señales de habilitación hacia sub-FSMs; las sub-FSMs devuelven flags (done, match, timeout).

Roles / ejemplos:

- **FSM Principal (Moore-like minimal)**: estados: INIT, WAIT_INPUT, EVALUATE, UPDATE_SCORE, CHECK_END, SWITCH_TURN, END.
- **SelectionFSM (Mealy)**: maneja debounce, traducción de switches a índice de carta, permite respuesta inmediata (por ejemplo, cuando pulsación → salida `index_valid`).
- **TimeoutFSM**: cuenta 15 s; en timeout emite `timeout_flag` y pide a RNG un índice.
- **RevealFSM**: controla la secuencia visual de revelado y ocultado de cartas (tiene temporizadores cortos).
- **ScoreFSM**: lleva contadores por jugador y señales a 7-seg.

Ventajas:

- Modular: cada sub-FSM pequeña, más fácil de testear y reutilizar.
- Más reactivo: ciertas salidas pueden generarse por Mealy (p. ej. `index_valid`) reduciendo latencia desde pulsación a acción.
- Facilita pruebas unitarias y depuración (testbench para cada sub-FSM y para la integración).
- Escala mejor (por ejemplo, si añades Arduino como segundo jugador, sólo cambias SelectionFSM/IF).

Desventajas:

- Requiere interfaz bien definida entre FSMs (handshakes), más señales de control.
- Riesgo de violar requisito “FSM debe generar únicamente señales de control” si sub-FSMs toman decisiones autónomas; hay que definir claramente: la FSM principal sigue siendo la que decide turnos y comparaciones — sub-FSMs sólo ejecutan tareas cuando la principal los habilita.
- Salidas Mealy pueden introducir *glitches* si no se sincronizan correctamente; requiere *flops* de sincronización.

2.4. Diseño elegido

En cuanto al diseño elegido se decide implementar la primera propuesta pues se considera que es más sencilla de implementar, los estados de la FSM están definidos y no requiere máquinas auxiliares. Además la propuesta 1 presenta una idea más clara de como se verían las cartas desplegadas en el monitor por medio del controlador VGA y se considera que es realizable con el tiempo que se dispone.